

HCLTech Agentic AI Hackathon 2026

June 8 – June 30, 2026 | Track 1: Industry / Business Transformation

RadReportAgent

Agentic AI for Automated Radiology Report Generation

Participant

Ashutosh Yadav
IIT Guwahati | HCL Technologies

Domain

Healthcare / Medical AI
OpenAI GPT-4o + Agents SDK

1. Project Overview

1.1 One-Line Pitch

"An agentic AI system that takes a chest X-ray + patient history and autonomously generates a structured radiology report, flags critical findings, and recommends follow-up actions — reducing report turnaround from 2 hours to under 2 minutes."

1.2 Problem Statement

Radiologists worldwide face an overwhelming backlog. In India alone, there is 1 radiologist per 100,000 population (WHO benchmark: 1 per 50,000). A single radiologist reviews 50–80 chest X-rays per day, writing structured reports for each — a repetitive, time-consuming process prone to fatigue-related errors.

- Report turnaround times of 2–6 hours delay clinical decisions
- Radiologist shortage in tier-2 and tier-3 cities creates diagnostic gaps
- Manual report writing consumes 30–40% of radiologist time
- Inconsistent report formats across hospitals hinder downstream analytics

1.3 Solution

RadReportAgent is an agentic AI system powered by OpenAI GPT-4o that autonomously:

1. Analyzes a chest X-ray image using multimodal vision AI
2. Extracts clinical findings (opacity, consolidation, effusion, cardiomegaly, etc.)
3. Classifies severity: Normal / Mild / Moderate / Critical
4. Fuses findings with patient history via tool-calling agents
5. Generates a structured radiology report in clinical SOAP format
6. Recommends follow-up actions (repeat imaging, specialist referral, urgent escalation)
7. Outputs a downloadable PDF report in seconds

1.4 Why This Wins

Layer	Tool / Technology	Notes
Real domain depth	Built on actual radiology + MIMIC-CXR research experience (PneumoFusionNet)	Not a toy demo
Quantified impact	60% reduction in report time, addresses WHO radiologist shortage gap	Judges want numbers
Truly agentic	GPT-4o calls 3 tools autonomously — not just a single prompt wrapper	Core judging criterion
Demo-ready	Streamlit UI: upload X-ray → report in <2 min — visual and impressive	Presentation matters
OpenAI compliant	Built on GPT-4o + OpenAI Agents SDK — mandatory requirement met	Zero disqualification risk

2. Rules & Compliance Checklist

2.1 Hackathon Rules (from FAQ)

Task	Timeline	Status
Use OpenAI technology (mandatory)	Required	✓ Done
Team size 1–5 members	Required	✓ Done
Submit project page on bemyapp platform	June 30	● Pending
All required fields filled + page published	June 30	● Pending
Demo a working prototype	Hackathon day	● Pending

2.2 Data Compliance

 MIMIC-CXR	RESTRICTED — PhysioNet DUA prohibits sharing in public demos or company platforms. DO NOT use MIMIC images in demo or submission.
 Use Instead	NIH Chest X-Ray Dataset (Kaggle) — fully public, 112,000+ images, no credentialing required. URL: kaggle.com/datasets/nih-chest-xrays

Your MIMIC-CXR experience (PneumoFusionNet research) informs your architecture and pitch credibility — just never show/submit actual MIMIC images. Mention your research background in the pitch as domain expertise context.

2.3 OpenAI Access Strategy

Task	Timeline	Status
Request via OneConnect: 'OpenAI Developer Platform'	ASAP	● Blocker
Personal OpenAI API key (backup — fully compliant)	Today	✓ Done
Azure OpenAI (AI-102 fallback)	If needed	● Pending

 Key Rule	FAQ says 'mandatory to build on any one of the OpenAI technologies' — using your personal OpenAI API key with GPT-4o fully satisfies this. HCL enterprise access is a convenience, not a hard gate.
---	---

3. Technical Architecture

3.1 System Architecture

RadReportAgent follows a modular agentic pipeline:

Input Layer	Chest X-ray (JPG/PNG upload) + Patient history form (age, symptoms, prior conditions, medications)
Vision Analysis	GPT-4o multimodal → extracts findings: opacities, consolidation, effusion, cardiomegaly, pneumothorax, masses
Agent Core	OpenAI Agents SDK orchestrates 3 tool calls in sequence based on findings
Tool 1	severity_classifier() → classifies as Normal / Mild / Moderate / Critical with confidence score
Tool 2	report_formatter() → generates structured SOAP/radiology report with findings, impression, comparison
Tool 3	followup_recommender() → recommends next steps: observation, repeat imaging, specialist referral, or urgent escalation
Output Layer	Structured radiology report (PDF download) + color-coded severity badge + follow-up action plan

3.2 Tech Stack

Layer	Tool / Technology	Notes
Agent Framework	OpenAI Agents SDK	Mandatory per hackathon rules
Vision / LLM	GPT-4o (multimodal)	Image + text in single call
Backend	Python + FastAPI	REST API for agent orchestration
Frontend	Streamlit	Fast to build, great for demos
Report Output	ReportLab / FPDF	PDF generation from agent output
Demo Data	NIH Chest X-Ray (Kaggle)	Fully public, no DUA required
Version Control	Git (dev branch)	Consistent with your workflow

3.3 Why Agentic (not just a prompt)

Judges will specifically evaluate whether the solution is truly agentic. RadReportAgent qualifies because:

- The agent autonomously decides which tools to call based on image findings

- Tool calls are sequential and conditional — severity determines report format and follow-up
- The agent can ask for clarification (e.g., missing patient history) before generating report
- Multi-step reasoning: vision → classification → formatting → recommendation — not a single GPT call

4. Day-by-Day Execution Plan

Week 1: June 8–14 — Foundation

Goal: Access + environment ready + core concept validated with GPT-4o vision.

Task	Timeline	Status
Submit OneConnect request for OpenAI Developer Platform	June 8	● Do today
Set up personal OpenAI API key as backup	June 8	● Do today
Create GitHub repo, project folder structure	June 8–9	● Pending
Download NIH Chest X-Ray sample (5–10 images from Kaggle)	June 9	● Pending
Test GPT-4o vision API with chest X-ray — validate it extracts findings	June 10–11	● Pending
Design agent tool schema (3 tools: severity, report, followup)	June 12–13	● Pending
FastAPI skeleton + Streamlit UI scaffold	June 13–14	● Pending

Week 2: June 15–21 — Build

Goal: End-to-end working prototype — upload X-ray → get report.

Task	Timeline	Status
Implement OpenAI Agents SDK agent loop	June 15–16	● Pending
Build severity_classifier() tool	June 16–17	● Pending
Build report_formatter() tool (SOAP format output)	June 17–18	● Pending
Build followup_recommender() tool	June 18–19	● Pending
Wire all tools into agent — test end-to-end	June 19–20	● Pending
Streamlit UI: X-ray upload + patient history form + report display	June 20–21	● Pending

Week 3: June 22–28 — Polish

Goal: Demo-ready, pitch-ready, submission-ready.

Task	Timeline	Status
Add PDF report download via ReportLab	June 22–23	 Pending
Add color-coded severity badge in UI (green/yellow/orange/red)	June 23	 Pending
Test with 5 different X-ray samples, fix edge cases	June 23–24	 Pending
Write submission page: problem → solution → impact → architecture	June 25–26	 Pending
Record 2–3 min demo video (screen record Streamlit walkthrough)	June 26–27	 Pending
Rehearse pitch: 2-minute verbal + demo flow	June 27–28	 Pending

June 29–30 — Submit

Task	Timeline	Status
Final QA: test all 3 agent tools, verify PDF output	June 29	 Pending
Publish project page on bemyapp platform	June 29–30	 Pending
Upload demo video to project page	June 29–30	 Pending
Submit before deadline — confirm confirmation email	June 30	 Pending

5. Code Scaffold — Start Here

5.1 Project Structure

```
RadReportAgent/  
  app/  
    main.py          # FastAPI entrypoint  
    agent.py         # OpenAI Agents SDK agent loop  
    tools.py         # 3 agent tools: severity, report, followup  
    report_gen.py    # PDF generation with ReportLab  
  ui/  
    streamlit_app.py # Frontend UI  
  data/  
    samples/         # NIH Chest X-Ray sample images  
  requirements.txt  
  README.md
```

5.2 requirements.txt

```
openai>=1.30.0  
fastapi  
uvicorn  
streamlit  
reportlab  
pillow  
python-multipart  
python-dotenv
```

5.3 Core Agent (agent.py) — Key Pattern

```
from openai import OpenAI  
  
client = OpenAI() # uses OPENAI_API_KEY from .env  
  
tools = [  
    {"type": "function", "function": {  
        "name": "severity_classifier",  
        "description": "Classify finding severity: Normal/Mild/Moderate/Critical",  
        "parameters": {"type": "object",  
            "properties": {"findings": {"type": "string"}},  
            "required": ["findings"]}  
    }},  
    # ... report_formatter, followup_recommender  
]
```

```
# Pass image + patient history to GPT-4o
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": image_b64_url}},
        {"type": "text", "text": patient_history_prompt}
    ]},
    tools=tools,
    tool_choice="auto"
)
```

6. Winning Pitch Guide

6.1 Submission Page Structure (bemyapp)

Structure your project page in this exact order:

8. Problem (2–3 sentences) — Radiologist shortage + report backlog. Include the stat: 1 radiologist per 100,000 in India.
9. Solution (3–4 sentences) — RadReportAgent: chest X-ray + history → structured report in 90 seconds via GPT-4o agents.
10. Architecture diagram — simple flowchart: Input → GPT-4o Vision → Agent Tools → PDF Report
11. Impact (bullet points) — quantified: 60% time reduction, 2 hrs → 2 min, scalable to rural hospitals
12. Demo video — 2–3 min Streamlit walkthrough: upload image, fill form, watch report generate
13. Tech stack — OpenAI Agents SDK, GPT-4o, FastAPI, Streamlit, Python

6.2 Verbal Pitch (2 Minutes)

0:00–0:20	Hook: 'Every day, 40 million chest X-rays are taken globally. Each needs a radiologist report. In India, 1 radiologist serves 100,000 people. The backlog is killing patients.'
0:20–0:50	Problem: 'Radiologists spend 40% of their time writing repetitive structured reports. Turnaround is 2–6 hours. Rural hospitals have no radiologist at all.'
0:50–1:20	Solution: 'RadReportAgent uses GPT-4o and the OpenAI Agents SDK to analyze a chest X-ray, classify findings, and generate a clinical-grade PDF report — in under 2 minutes.'
1:20–1:45	Demo: Run the live Streamlit demo — upload NIH sample X-ray, show agent tool calls firing, show PDF report generated.
1:45–2:00	Close: 'This is not a chatbot wrapper. It calls 3 tools autonomously, generates structured clinical output, and is deployable in any hospital with an internet connection.'

6.3 Judge Questions — Prep

Task	Timeline	Status
Why is this agentic and not just a prompt?	Prep answer	3 tools called autonomously
Why GPT-4o and not an open-source model?	Prep answer	Accuracy + hackathon compliance
How does it handle wrong diagnoses?	Prep answer	Confidence scores + human-in-loop disclaimer
What's the business model?	Prep answer	SaaS per-report pricing for hospitals
Is patient data safe?	Prep answer	No PII stored; demo uses public dataset

7. Resources & Links

7.1 Hackathon Links

- Event platform: <https://hcltech-codex-agenticaihack.bemyapp.com>
- Register teammates: <https://hcltech-codex-agenticaihack.bemyapp.com/#/register>
- Teams Build Hub channel (for HCL access, updates, office hours)
- OneConnect (HCL OpenAI access request): <https://oneconnect.hcltech.com/esc>

7.2 Technical Resources

- OpenAI Agents SDK docs: <https://platform.openai.com/docs/guides/agents>
- GPT-4o vision guide: <https://platform.openai.com/docs/guides/vision>
- NIH Chest X-Ray dataset: <https://www.kaggle.com/datasets/nih-chest-xrays/data>
- ReportLab PDF docs: <https://docs.reportlab.com>
- Streamlit docs: <https://docs.streamlit.io>

7.3 Your Domain Advantage

Research

PneumoFusionNet (IIT Guwahati, Prof. Ritika Jain) — Phase 1 complete: ResNet50 + GCSA + DSC on MIMIC-CXR. Use this experience in your pitch for credibility. Do NOT use MIMIC data in demo.

Certifications

AI-102 Azure AI Engineer Associate — demonstrates enterprise AI deployment knowledge. Mention in pitch if asked about production deployment.